



ALLIANCE
UNIVERSITY

*Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi*

DATABASE MANAGEMENT SYSTEM

CAB BOOKING MANAGEMENT SYSTEM

A relational database design for managing customers, drivers, cabs, bookings, trips, payments and ratings.

BACHELORS OF COMPUTER APPLICATION- III

NAME: PREKSHA V

REGISTER NUMBER: 2511021240063

SUBMITTING TO,

RENITA JOY R

DBMS ASSIGNMENT

Relational Database • ER Modeling • Keys & Relationships

CONTENTS

A structured guide to the ten-page database design document.

01	Cover Page	Cab Booking Management System
02	Index	Contents and document structure
03	Introduction	Background, purpose and problem statement
04	Application Overview	Objectives, users and booking lifecycle Customer,
05	Database Tables — Part I	Driver, Cab and Booking
06	Database Tables — Part II	Trip, Payment, Rating and Driver_Location
07	Relationships	Keys, cardinalities and table connections
08	ER Relationship Diagram	Visual representation of the complete database
09	System Workflow	Example booking and movement of data
10	Conclusion	DBMS concepts demonstrated and final summary

DESIGN PRINCIPLE

The database is focused on information required to operate a cab-booking service. Each table represents a meaningful business requirement rather than being added only to increase the count.

INTRODUCTION

Cab-booking platforms connect passengers with drivers through a sequence of database-driven operations. A customer creates a booking, the system identifies an available driver and cab, the journey is recorded, payment is processed, and feedback may be collected after the ride.

PROBLEM STATEMENT

Managing these activities without a structured database can cause duplicated information, inconsistent booking records, difficulty tracking trips and payments, and poor access to driver availability and location data. A relational database provides a systematic way to store and connect this information.

PURPOSE OF THE DATABASE

- Maintain accurate customer and driver information.
- Store cab and vehicle details.
- Record every booking and its current status.
- Track trips, distance and fare.
- Maintain payment records.
- Capture customer ratings and reviews.
- Store driver location updates for matching and availability.

CORE IDEA

The database transforms a real-world cab journey into connected records. Related information is separated into logical tables and connected through keys, improving organization, consistency and retrieval.

APPLICATION OVERVIEW

MAIN OBJECTIVE

The main objective is to provide a database structure supporting the complete cab-booking lifecycle from customer request to completed trip, payment and optional rating.

MAIN USERS / ACTORS

Actor	Role in the system
Customer	Books a cab, enters pickup/drop locations and may rate the ride.
Driver	Accepts bookings, completes trips and provides availability/location.
System	Matches bookings with available drivers/cabs and maintains records.

TYPICAL BOOKING LIFECYCLE

Step	Stage	Description
1	Request	Customer enters pickup and destination.

Step	Stage	Description
2	Assignment	System assigns an available driver and cab.

Step	Stage	Description
3	Trip	Ride starts and journey details are recorded.

Step	Stage	Description
4	Completion	Trip ends and fare is finalized.

Step	Stage	Description
5	Payment	Customer pays using a payment method.

Step	Stage	Description
6	Feedback	Customer may provide a rating and review.

DATABASE TABLES — PART I

CUSTOMER

Column	Type	Description
Customer_ID (PK)	INT	Unique customer identifier
Name	VARCHAR	Customer name
Phone	VARCHAR	Contact number
Email	VARCHAR	Email address
Address	VARCHAR	Customer address

Purpose: Stores passengers who use the cab-booking application.

DRIVER

Column	Type	Description
Driver_ID (PK)	INT	Unique driver identifier
Name	VARCHAR	Driver name
Phone	VARCHAR	Contact number
License_Number	VARCHAR	Driving license number
Status	VARCHAR	Available / Busy / Offline

Purpose: Stores registered drivers and their current availability.

CAB

Column	Type	Description
Cab_ID (PK)	INT	Unique cab identifier
Driver_ID (FK)	INT	Assigned driver
Vehicle_Number	VARCHAR	Registration number
Cab_Type	VARCHAR	Sedan / SUV / Hatchback
Model	VARCHAR	Vehicle model
Status	VARCHAR	Available / Booked / Maintenance

Purpose: Stores vehicles used to provide cab services.

BOOKING

Column	Type	Description
Booking_ID (PK)	INT	Unique booking identifier
Customer_ID (FK)	INT	Customer who booked
Driver_ID (FK)	INT	Assigned driver
Cab_ID (FK)	INT	Assigned cab
Booking_Date	DATETIME	Date and time of booking
Pickup_Location	VARCHAR	Pickup point
Drop_Location	VARCHAR	Destination
Booking_Status	VARCHAR	Pending / Confirmed / Cancelled / Completed

Purpose: Records each cab reservation and its assignment details.

DATABASE TABLES — PART II

TRIP

Column	Type	Description
Trip_ID (PK)	INT	Unique trip identifier
Booking_ID (FK)	INT	Associated booking
Start_Time	DATETIME	Trip start time
End_Time	DATETIME	Trip end time
Distance	DECIMAL	Distance travelled
Fare	DECIMAL	Final trip fare
Trip_Status	VARCHAR	Started / Completed / Cancelled

Purpose: Stores actual journey details after a booking is made.

PAYMENT

Column	Type	Description
Payment_ID (PK)	INT	Unique payment identifier
Booking_ID (FK)	INT	Associated booking
Amount	DECIMAL	Amount paid
Payment_Method	VARCHAR	UPI / Card / Cash
Payment_Date	DATETIME	Payment date
Payment_Status	VARCHAR	Successful / Pending / Failed

Purpose: Records the payment associated with a booking.

RATING

Column	Type	Description
Rating_ID (PK)	INT	Unique rating identifier
Booking_ID (FK)	INT	Associated booking
Customer_ID (FK)	INT	Customer giving rating
Driver_ID (FK)	INT	Driver being rated
Rating	INT	Rating from 1 to 5
Review	VARCHAR	Customer feedback

Purpose: Stores customer feedback for completed rides.

DRIVER_LOCATION

Column	Type	Description
Location_ID (PK)	INT	Unique location record
Driver_ID (FK)	INT	Driver being tracked
Latitude	DECIMAL	Latitude coordinate
Longitude	DECIMAL	Longitude coordinate
Updated_Time	DATETIME	Last location update

Purpose: Stores driver location updates for matching and availability.

RELATIONSHIPS BETWEEN TABLES

Primary keys uniquely identify records, while foreign keys connect records across related tables. The following relationships define how the cab-booking database is connected.

Customer → Booking	1 : M	One customer can make many bookings.
Driver → Booking	1 : M	One driver can complete many bookings over time.
Driver → Cab	1 : 1* 1	One driver is associated with one cab in this simplified model.
Cab → Booking	: M	One cab can be used for many bookings over time.
Booking → Trip	1 : 1	A booking produces one trip record.
Booking → Payment	1 : 1	A booking has a corresponding payment record.
Booking → Rating	1 : 0..1	A completed booking may receive one rating.
Driver → Driver_Location	1 : M	A driver's location can be updated many times.
Customer → Rating	1 : M	A customer can submit multiple ratings over time.
Driver → Rating	1 : M	A driver can receive multiple ratings from rides.

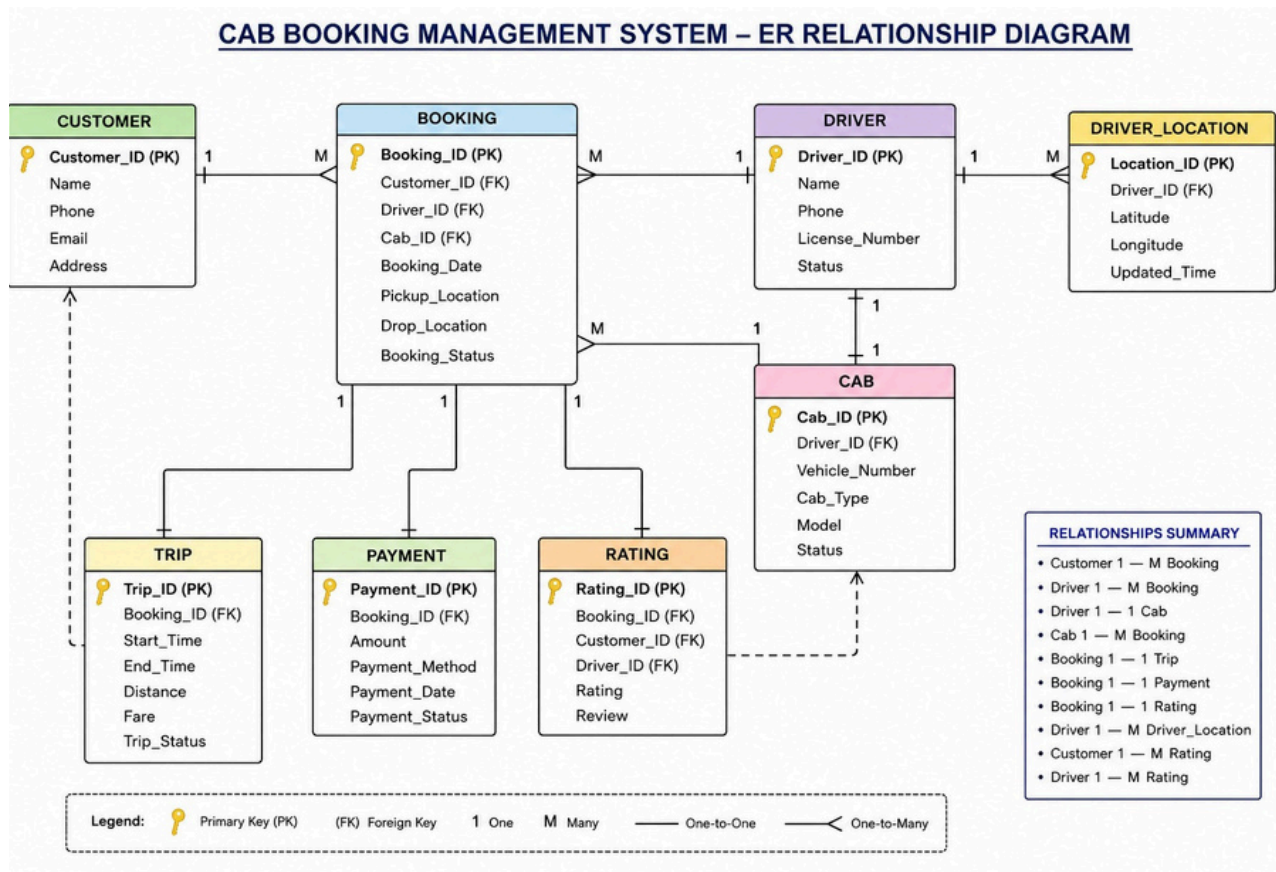
KEY DBMS CONCEPT — CARDINALITY

1:M means one record on the first side can be related to many records on the second side. 1:1 means one record corresponds to one record. The Rating relationship is optional because a customer may finish a ride without submitting feedback.

Note: Driver → Cab 1:1 is a simplified academic assumption. A production system could model vehicle assignment differently if drivers use multiple vehicles.

ER RELATIONSHIP DIAGRAM

The diagram below visually represents the entities , primary/foreign keys and cardinalities in the proposed database.



ER relationship structure for the Cab Booking Management System.

SYSTEM WORKFLOW & EXAMPLE

Consider a customer booking a cab from Alliance University to Electronic City. The example shows how one real-world transaction creates connected database records.

Step	Table	Illustrative record
1. Customer	Customer	Customer_ID = 101; Name = Rahul
2. Booking	Booking	Booking_ID = 5001; Customer_ID = 101; Pickup = Alliance University
3. Assignment	Driver + Cab	Driver_ID = 205; Cab_ID = 301; Status = Confirmed
4. Trip	Trip	Trip_ID = 7001; Distance = 18.5 km; Fare = ₹350
5. Payment	Payment	Payment_ID = 8001; Method = UPI; Status = Successful
6. Feedback	Rating	Rating_ID = 9001; Rating = 5; Review = Good ride

HOW THE RECORDS CONNECT

- Customer_ID links the customer to the booking.
- Driver_ID and Cab_ID connect the booking to its assigned driver and vehicle.
- Booking_ID links the booking to its trip and payment records.
- Booking_ID, Customer_ID and Driver_ID associate a rating with the correct ride and participants.
- Driver_ID connects repeated location updates to the correct driver.

WHY THIS STRUCTURE WORKS

A single booking can be retrieved together with its customer, assigned driver, cab, trip, payment and rating by following foreign-key relationships. This makes the design practical for searching, reporting and transaction management.

CONCLUSION

The Cab Booking Management System demonstrates how a real-world transportation application can be represented using a relational database. The design separates customer, driver, cab, booking, trip, payment, rating and location information into focused tables.

DBMS CONCEPTS DEMONSTRATED

Concept	How it appears in this system
Primary Key	Uniquely identifies records such as Customer_ID and Booking_ID.
Foreign Key One-to-	Connects tables, such as Booking.Customer_ID → Customer.Customer_ID.
Many One-to-One	One customer can make many bookings; one driver can handle many bookings.
Optional Relationship	A booking is associated with one trip and one payment in the simplified model.
Data Organization	A booking may or may not have a rating.
	Separate tables reduce unnecessary duplication and improve clarity.

FINAL SUMMARY

The database is designed to solve the actual information-management problem of a cab-booking service. It provides a logical structure for storing data and demonstrates how relationships between real-world entities can be translated into relational database tables.

“A well-designed database does not simply store data — it organizes relationships so the application can work intelligently.”